AWS Multi-Account EC2 Inventory Automation

Technical Architecture & Operational Documentation

1. Executive Summary & Overview

This automated solution scans across multiple AWS accounts and regions to build a consolidated, professionally formatted Microsoft Excel (.xlsx) inventory report of all running and stopped EC2 instances.

The entire workflow executes statelessly inside an AWS Lambda function written in Python. It leverages AWS Security Token Service (STS) to assume IAM roles across secondary accounts, extracts 30-day historical CPU utilization metrics from Amazon CloudWatch, dynamically auto-formats the spreadsheet layout (including gridlines, dynamic column widths, and universal center alignment) using openpyxl, and delivers the finalized workbook as an email attachment via Amazon SES.

Additionally, the solution includes a passive auditing mechanism that identifies instances launched by Auto Scaling Groups (ASGs) that lack a Name tag. These anomalies are grouped into a dedicated secondary table placed at the bottom of each account's worksheet for manual review.

2. System Architecture & Workflow

The architecture follows an event-driven, cross-account fan-out model executed entirely in memory:

PHASE 1: HOST ACCOUNT EXECUTION & INITIATION	
[EventBridge Schedule] ➔ [AWS Lambda Function] • Reads environment variables (Target Accounts, Roles, SES Settings) • Initiates STS assume-role handshake across target accounts via boto3	
TARGET ACCOUNT A Role: EC2InventoryRole • ec2:DescribeInstances • cloudwatch:GetMetricStatistics	**TARGET ACCOUNT B / N** Role: EC2InventoryRole • ec2:DescribeInstances • cloudwatch:GetMetricStatistics
⬇ Returns Aggregated Metadata & CPU Utilization Metrics ⬇	
PHASE 2: CONSOLIDATION, FORMATTING & EMAIL DISPATCH 1. Builds multi-sheet Excel workbook in-memory using openpyxl Buffer (io.BytesIO) 2. Formats gridlines, universal center alignment, titles, and dynamic column widths 3. Constructs HTML summary body and attaches Base64-encoded workbook via MIME 4. Dispatches email to recipient list using Amazon SES (ses:SendRawEmail)	

3. Prerequisites & Environment Setup

3.1 Environment Variables (Lambda Configuration)

Environment Variable	Description	Example Value
SENDER_EMAIL	Verified SES sender email address	reports@yourdomain.com
RECIPIENT_EMAILS	Comma-separated list of recipient emails	team@yourdomain.com, audit@yourdomain.com
TARGET_ACCOUNTS	Comma-separated list of 12-digit AWS Account IDs	111122223333, 444455556666
SES_REGION	AWS Region where SES identity is verified	ap-south-1
ROLE_NAME	Name of cross-account IAM role in target accounts	EC2InventoryRole

3.2 Python Dependencies & Deployment Package

- **boto3:** AWS SDK for Python (included natively in the AWS Lambda runtime).
- **pandas:** High-performance data manipulation and DataFrame structurer.
- **openpyxl:** Low-level spreadsheet engine for styling borders, alignments, dynamic column auto-sizing, and cell merging.

4. Cross-Account Connectivity & IAM Handshake

Cross-account access relies on a two-way IAM trust handshake utilizing temporary security tokens provided by AWS STS.

4.1 Host Account (Lambda Execution Role)

The Lambda function's IAM Role requires outbound permission to assume target roles and dispatch emails.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "AllowCrossAccountRoleAssumption",
            "Effect": "Allow",
            "Action": "sts:AssumeRole",
            "Resource": "arn:aws:iam::*:role/EC2InventoryRole"
        },
        {
            "Sid": "AllowSESEmailDispatch",
            "Effect": "Allow",
            "Action": "ses:SendRawEmail",
            "Resource": "*"
        }
    ]
}
```

4.2 Target Account (EC2InventoryRole)

Every secondary target account must contain an IAM role named EC2InventoryRole.

- 1. Trust Relationship Policy (Who can assume this role):

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Principal": {
                "AWS": "arn:aws:iam::<HOST_ACCOUNT_ID>:role/<LAMBDA_EXECUTION_ROLE_NAME>"
            },
            "Action": "sts:AssumeRole"
        }
    ]
}
```

- 2. Permissions Policy (What the role can do inside the target account):
  -> Provide read only access in the target account for easier setup

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "ec2:DescribeInstances",
                "ec2:DescribeRegions",
                "cloudwatch:GetMetricStatistics"
            ],
            "Resource": "*"
        }
    ]
}
```

5. Core Logic & Data Engine Specifications

5.1 Multi-Region Processing Loop

For every target account specified in TARGET_ACCOUNTS:

- 1. Local vs. Cross-Account Routing: The Lambda checks if the target Account ID matches the caller identity. If local, it uses local credentials; if external, it calls sts.assume_role().
- 2. Region Discovery: Calls ec2.describe_regions() to discover all active regions in the target account automatically.
- 3. Paginator Processing: Iterates through every region using ec2.get_paginator('describe_instances') to avoid payload truncation errors.

5.2 Metrics Aggregation (CloudWatch)

For every discovered instance ID, cloudwatch.get_metric_statistics() queries the AWS/EC2 namespace for CPUUtilization over the last 30 days (timedelta(days=30)). It calculates the arithmetic mean across returned daily datapoints and formats the result to two decimal places (e.g., 4.25%).

5.3 Passive Auditing Logic (Unnamed ASG Instances)

The script performs a passive evaluation of instance tags without modifying live infrastructure:

- Detection Trigger: If an instance has aws:autoscaling:groupName present (meaning it was spun up by an ASG) AND its Name tag is either completely missing or empty/whitespace ("").
- Handling: The main table displays the instance name as empty/blank "". Simultaneously, the instance metadata is appended to a separate array unnamed_asg_inventory to build a secondary audit table.

6. Excel Formatting & Styling Engine

The script constructs a single .xlsx workbook where each AWS Account receives its own dedicated Worksheet tab named after its Account ID.

6.1 Applied Styling Rules (openpyxl)

- Title Banners: Row 1 contains a bold, centered header displaying AWS Account ID: <ACCOUNT_ID>, dynamically merged across all active columns (worksheet.merge_cells).
- Universal Center Alignment: Every populated cell in both tables is assigned Alignment(horizontal='center', vertical='center').
- Perimeter Gridlines: A uniform thin border (Border(left=Side(style='thin'), right=Side(style='thin'), top=Side(style='thin'), bottom=Side(style='thin'))) is applied across all active data grid cells.
- Dynamic Column Auto-Sizing: The script loops through every column, calculates the maximum string length of the cell values, and sets worksheet.column_dimensions[col_letter].width = max(max_len + 4, 12).
- Exclusion Rule: Merged title rows (Row 1 and the Sub-title Row) are explicitly skipped during width calculation to prevent Column A from warping out of proportion.

7. Email Compilation & Delivery

- Subject Line: Formatted dynamically with the current execution month: AWS EC2 Inventory - <FULL_MONTH_NAME> (e.g., AWS EC2 Inventory - July).
- Email Body: Structured as a clean HTML email styled with AWS brand colors (#232F3E dark grey and #FF9900 orange accents), summarizing total accounts scanned, total EC2 instances counted, and the UTC generation timestamp.
- Attachment Streaming: To operate within Lambda's read-only filesystem, the workbook is compiled entirely in memory using io.BytesIO(). It is encoded into Base64 using MIMEBase and attached as EC2_Inventory_<YYYYMMDD>.xlsx.

8. Performance, Limits & Scaling Guidelines

8.1 Resource Configuration Table

Setting	Standard Default	Recommended Setting	Reason
Timeout	3 seconds	15 minutes (900s)	CloudWatch API statistics calls are synchronous. Processing 2,000 instances sequentially takes ~5–8 minutes.

Memory (RAM)	128 MB	1024 MB – 2048 MB	Compiling large dataframes and building multi-sheet openpyxl objects in-memory requires dedicated RAM.
Ephemeral Storage	512 MB	512 MB (Default)	File processing occurs entirely in-memory using io.BytesIO().

8.2 API Rate Limiting & Throttling Mitigation

AWS CloudWatch enforces rate limits on GetMetricStatistics. If scanning thousands of instances triggers ThrottlingException errors, configure boto3 retry behavior by setting the AWS_RETRY_MODE environment variable on the Lambda function to adaptive.